Group 4

Efficient LLM Serving

Problem and Proposed Approach

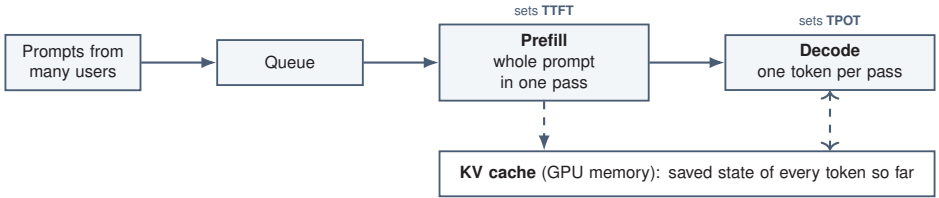
Project 24: Quantisation, Prefix Caching and Speculative Decoding

Vishank Singh AI23BTECH11032

Anup Chavan CS23BTECH11005

16 October 2026

LLM serving basics



- **TTFT**: time to first token (queue + prefill). **TPOT**: time per output token after that.
- **Throughput**: requests finished per second.
- Decode re-reads all weights for every token, so it is limited by memory bandwidth. Prefill is limited by compute.

Problem

- Default vLLM on one GPU is slower than it could be. Known fixes are smaller weights, a smaller KV cache, reuse of shared prompt prefixes and several tokens per pass.
- Quantisation frees memory, but a draft model for speculative decoding uses memory and GPU time.
- Whether a fix helps depends on load, prompt length and prompt reuse.

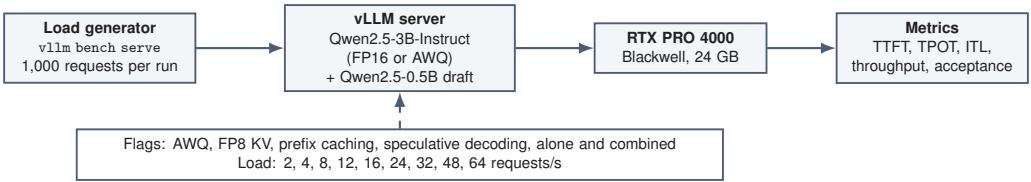
Milestone	Scope	Expected outcome
M1	Quantised weights, FP8 KV cache, batching on one GPU	Measured gain over default vLLM at a latency target, quality kept
M2	Prefix caching, speculative decoding	Conditions where each helps; load where speculation stops paying
M3	All combined	Contribution of each technique; limiting resource; regimes where the setup is wrong

Techniques

Technique	Idea	Bottleneck	vLLM flag
AWQ	4-bit weights instead of 16-bit	Memory bandwidth in decode	`-quantization awq`
FP8 KV cache	8-bit KV cache; about twice as many tokens fit	KV-cache capacity	`-kv-cache-dtype fp8`
Prefix caching	Reuse KV cache of a prompt prefix seen before	Prefill compute	`-enable-prefix-caching`
Speculative decoding	0.5B draft guesses 5 tokens; 3B model verifies them in one pass	Idle compute in decode	`-speculative-config`

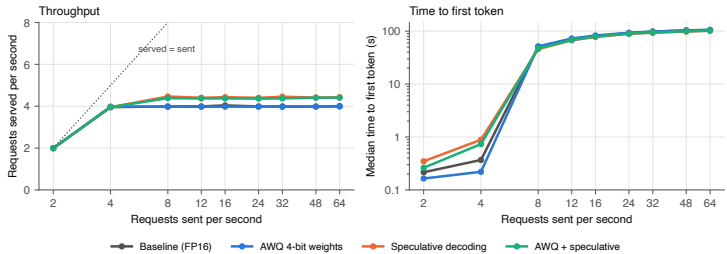
- Speculative decoding keeps the output unchanged; the 3B model verifies every token.
- AWQ changes the weights, so output quality needs a separate check.

Approach



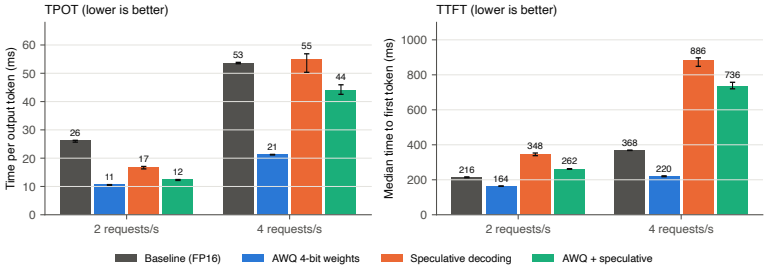
- Each technique alone, then in combination, to separate individual effects from interactions.
- Load from light to far past capacity.
- Workload: 1,024 prompt + 128 output tokens per request.
- 6 repeats per setting; about 560 runs over 4 days.

Results: throughput limit



- Up to 4 requests/s, every request is served and TTFT is 0.2–0.9 s.
- Above that, the server finishes 4.0–4.4 requests/s and the rest queue; TTFT reaches 50–106 s.
- No technique moves this limit much. Likely cause: prefill compute (about 4,000 prompt tokens/s), still to be profiled.

Results: below the limit



Median of 4–6 runs; bars show the range. 1,024 prompt + 128 output tokens.

- AWQ: 2.5× lower TPOT (26 → 11 ms) and lower TTFT at both loads.
- Speculative decoding: 1.6× lower TPOT at 2 requests/s, none at 4; higher TTFT at both.

Contribution of each technique

TPOT speed-up over the FP16 baseline (higher is better); TTFT change in brackets.

Configuration	2 requests/s	4 requests/s	Overloaded, 16–64/s
Baseline (FP16, vLLM defaults)	1.00×	1.00×	4.0 requests/s served
AWQ	**2.47**× (TTFT −24%)	**2.52**× (TTFT −40%)	4.0
Speculative decoding	1.56× (TTFT +61%)	0.98× (TTFT +141%)	**4.4** (+11%)
AWQ + speculative	2.11× (TTFT +21%)	1.22× (TTFT +100%)	4.4

- AWQ alone (2.5×) beats AWQ with speculation (2.1× and 1.2×). With AWQ each 3B pass is cheap, so 5 draft steps cost more than they save; 55% of drafted tokens are accepted.
- Speculation stops paying between 2 and 4 requests/s. When overloaded it adds 11% throughput, but TTFT is over a minute anyway.

Status

Item	Status	Note
M1: AWQ	**Done**	2.5× lower TPOT below 4 requests/s; quality not measured
M1: FP8 KV cache	Blocked	Needs CUDA toolkit
M1: batching	Not started	Single GPU, so no sharding question
M2: speculative decoding	**Done**	Break-even between 2 and 4 requests/s
M2: prefix caching	Next	Needs prompts with shared prefixes
M3: attribution	First version	One prompt length only

- Below the limit, AWQ alone was the best setup we tested. Speculative decoding helps only at light load.

Limitations and next steps

- **Limitations:** one prompt length (1,024 tokens); no shared prefixes; open-loop load far past capacity; AWQ quality not checked; one GPU, one model size.

Next two weeks	Then	Before the final review
Prompt lengths 512–6,144	Fixed-concurrency sweep	Attribution across regimes
Prefix caching off vs. on, shared prefixes	2–3 draft tokens; other drafters	Best setup per regime
CUDA toolkit; FP8 KV cache	Goodput at latency target	Profiling of the limit
AWQ quality on a small benchmark	Batch-size and scheduling tuning	Final plots and scripts

- **Latency target (M1):** TTFT under 1 s and TPOT under 50 ms, reported as goodput (requests/s meeting both).

Thank You